Министерство образования Республики Беларусь
УО “Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «СПП»
Вариант 9

Выполнила:
студент 3 курса
группы ПО-12
Кравченко М.А.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API.

Задание.

9. Автоматизированный мониторинг активности в GitHub-репозитории

Условие:

Напишите Python-скрипт, который анализирует активность в указанном GitHub-репозитории и отслеживает:

1. Новые коммиты
2. Созданные и закрытые issues
3. Открытые и закрытые pull requests
4. Новых контрибьюторов
5. Упоминания в комментариях (@username)
6. Количество звёзд и форков

Скрипт должен уметь:

- Подключаться к GitHub API и получать последние изменения в репозитории.
- Фильтровать события по времени (например, за последние 24 часа).

Код программы:

```
"""
Автоматизированный мониторинг активности в GitHub-репозитории.
"""

from datetime import datetime, timedelta, timezone

import requests


class GitHubMonitor:
    """Класс для мониторинга активности в GitHub-репозитории."""

    def __init__(self, repo_name: str, token: str = None):
        """Инициализация монитора."""
        self.repo_name = repo_name
        self.base_url = "https://api.github.com"
        self.headers = {"Accept": "application/vnd.github.v3+json"}
        if token:
            self.headers["Authorization"] = f"token {token}"
        self.start_date = None
        self.end_date = None
        self.token = token

    def _parse_date(self, date_str: str) -> datetime:
        """Преобразовать строку даты из GitHub в datetime."""
        return datetime.fromisoformat(date_str.replace("Z", "+00:00"))

    def set_time_range(self, hours: int):
        """Установить временной диапазон для мониторинга."""
        self.end_date = datetime.now(timezone.utc)
        self.start_date = self.end_date - timedelta(hours=hours)

    def _make_request(self, url: str, params: dict = None) -> list:
        """Сделать запрос с обработкой ошибок."""
        try:
```

```
        response = requests.get(url, headers=self.headers, params=params, timeout=30)

        if response.status_code == 403 and "rate limit" in str(response.text):
            print("  Превышен лимит запросов. Используйте GitHub токен.")
            return []

        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"  Ошибка: {e}")
        return []

def get_commits(self) -> list:
    """Получить новые коммиты."""
    url = f"{self.base_url}/repos/{self.repo_name}/commits"
    params = {"since": self.start_date.isoformat(), "per_page": 100}
    return self._make_request(url, params)

def get_issues(self) -> tuple:
    """Получить созданные и закрытые issues."""
    url = f"{self.base_url}/repos/{self.repo_name}/issues"
    params = {"state": "all", "per_page": 100, "since": self.start_date.isoformat()}
    issues = self._make_request(url, params)

    if not issues:
        return [], []

    created = []
    closed = []
    for issue in issues:
        if "pull_request" in issue:
            continue
        created_at = self._parse_date(issue["created_at"])
        if created_at >= self.start_date:
            created.append(issue)
        if issue.get("closed_at"):
            closed_at = self._parse_date(issue["closed_at"])
            if closed_at >= self.start_date:
                closed.append(issue)
    return created, closed

def get_pull_requests(self) -> tuple:
    """Получить открытые и закрытые pull requests."""
    url = f"{self.base_url}/repos/{self.repo_name}/pulls"
    params = {"state": "all", "per_page": 100}
    pulls = self._make_request(url, params)

    if not pulls:
        return [], []

    opened = []
    closed = []
    for pull in pulls:
        created_at = self._parse_date(pull["created_at"])
        if created_at >= self.start_date:
            opened.append(pull)
        if pull.get("closed_at"):
            closed_at = self._parse_date(pull["closed_at"])
            if closed_at >= self.start_date:
                closed.append(pull)
    return opened, closed

def get_contributors(self) -> list:
    """Получить новых контрибьюторов."""
    url = f"{self.base_url}/repos/{self.repo_name}/commits"
    params = {"since": self.start_date.isoformat(), "per_page": 100}
    commits = self._make_request(url, params)
```

```
    if not commits:
        return []

    contributors = set()
    for commit in commits:
        author = commit.get("author")
        if author and author.get("login"):
            contributors.add(author["login"])
    return list(contributors)

def get_stars_and_forks(self) -> tuple:
    """Получить количество звезд и форков."""
    url = f"{self.base_url}/repos/{self.repo_name}"
    data = self._make_request(url)
    if data:
        return data.get("stargazers_count", 0), data.get("forks_count", 0)
    return 0, 0

def get_mentions(self) -> list:
    """Получить упоминания пользователей в комментариях."""
    mentions = set()
    url = f"{self.base_url}/repos/{self.repo_name}/issues/comments"
    params = {"since": self.start_date.isoformat(), "per_page": 100}
    comments = self._make_request(url, params)

    if not comments:
        return []

    for comment in comments:
        body = comment.get("body", "")
        for word in body.split():
            if word.startswith("@"):
                username = word[1:].strip(".,!?;:")
                if username:
                    mentions.add(username)
    return list(mentions)

def run_monitoring(self) -> dict:
    """Запустить мониторинг и вернуть статистику."""
    hours = int((self.end_date - self.start_date).total_seconds() / 3600)
    print(f"\nМониторинг '{self.repo_name}' за {hours} ч")
    print("-" * 50)

    commits = self.get_commits()
    print(f"Коммитов: {len(commits)}")

    issues_created, issues_closed = self.get_issues()
    print(f"Issues: создано {len(issues_created)}, закрыто {len(issues_closed)}")

    pulls_opened, pulls_closed = self.get_pull_requests()
    print(f"Pull requests: открыто {len(pulls_opened)}, закрыто {len(pulls_closed)}")

    new_contributors = self.get_contributors()
    if new_contributors:
        print(f"Новые контрибьюторы: {len(new_contributors)}")
        names = ", ".join(f"@{c}" for c in new_contributors[:5])
        print(f"  {names}")
        if len(new_contributors) > 5:
            print(f"  и еще {len(new_contributors) - 5}")

    mentions = self.get_mentions()
    if mentions:
        print(f"Упоминания: {len(mentions)}")
        names = ", ".join(f"@{m}" for m in mentions[:5])
        print(f"  {names}")
        if len(mentions) > 5:
            print(f"  и еще {len(mentions) - 5}")
```

```
        stars, forks = self.get_stars_and_forks()
        print(f"Звёзды: {stars}, Форки: {forks}")

        return {
            "commits": len(commits),
            "issues_created": len(issues_created),
            "issues_closed": len(issues_closed),
            "pulls_opened": len(pulls_opened),
            "pulls_closed": len(pulls_closed),
            "new_contributors": new_contributors,
            "mentions": mentions,
            "stars": stars,
            "forks": forks,
        }


def main():
    """Основная функция."""
    print("=" * 60)
    print("GitHub МОНИТОРИНГ")
    print("=" * 60)

    repo = input("Введите репозиторий (owner/repo): ").strip()
    if not repo:
        repo = "django/django"

    hours = int(input("Введите часы (по умолч. 24): ") or "24")

    print("\nБез токена - 60 запросов/час. С токеном - 5000 запросов/час.")
    token = input("GitHub токен (Enter если нет): ").strip()
    if not token:
        token = None

    monitor = GitHubMonitor(repo, token)
    monitor.set_time_range(hours)
    monitor.run_monitoring()


if __name__ == "__main__":
    main()
```

```
============================================================
GitHub МОНИТОРИНГ
============================================================
Введите репозиторий (owner/repo): kroschenko/spp_po12
Введите часы (по умолч. 24): 72
GitHub токен (Enter если нет): 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

Мониторинг 'kroschenko/spp_po12' за 72 ч
--------------------------------------------------
Коммитов: 22
Issues: +0 / -0
Pull requests: +11 / -9
Новые контрибьюторы: 7
  @Anastasia1Kulick, @kayosheesh, @Enixfai, @DakariLuin, @Parf1x
  и еще 2
Звёзды: 0, Форки: 24
```

Вывод: научилась работать с Github API, приобрела практические навыки написания программ для работы с REST API или GraphQL API.